Group 2: Test Plan

Project Title: Smart Mock Interview Assistant: An NLP-Based System for Evaluating Technical and Behavioural Interview Responses

Project Stage	Final submission	**Prepared by**	Test Plan Manager
Document Version	V3.0	**Team GitHub repository:**	https://github.com/Professional-Team-Project

Purpose: This test plan defines how the Smart Mock Interview Assistant is ready for final demonstration and User Acceptance Testing (UAT). It covers data quality, pipeline reliability, model scoring behaviour, feedback quality, API stability, frontend usability, report delivery, deployment readiness, and final sign-off evidence.

Research question: Can neural NLP models accurately evaluate technical interview responses and provide personalized feedback in a mock interview simulator for technology-related roles?

1. Scope of Application

Area	**Current implementation to test**	**Evidence expected**
Main Flask app	Serves frontend and backend together; exposes health, info, roles, domains, models, interview, report, PDF and email endpoints.	Successful local or deployed smoke-test results.
Frontend	Chat-style interview interface with role selection, custom domain flow, model selector, answer submission, repeat, skip, final report, PDF and email actions.	Completed UAT walkthroughs and participant feedback forms.
Scoring models	Siamese Bi-LSTM, Sentence-BERT and TF-IDF + Grammar available through one model registry.	Model score checks showing valid score ranges and sensible relative behaviour.
Interview progression	BERT/TF-IDF use adaptive Easy/Medium thresholds; Bi-LSTM uses fixed Easy/Medium/Hard curriculum.	Round progression tests for each model type.
Report delivery	Background report generation, saved JSON report, downloadable PDF, optional email delivery through SMTP configuration.	Report generated, PDF opened/downloaded, email path tested or limitation documented.
Comparison tool	Separate local Flask app on port 5050 for single-answer scoring and topic comparison across all three models.	Screenshots or notes from comparison-tool checks.

Table 1: Scope of testing

In scope: dataset quality checks; pre-processing tests; baseline, neural and transformer-based model testing; feedback quality checks; app/UAT testing; code review and reproducibility checks.

Out of scope: live voice/video analysis, facial emotion recognition, production-scale deployment, and replacing human interviewers or recruiters.

2. Coding Standards for Notebooks and Scripts

- Use clear, descriptive file names, for example data_cleaner.py, data_transformation.py, interview_session.py, with exploratory notebooks such as questions_eda.ipynb kept in a separate notebooks folder.
- Follow PEP 8 guidelines where possible, use meaningful function names, and add docstrings to reusable functions such as clean_text(), evaluate_model() and calculate_overall_score().
- Add markdown explanations in notebooks to improve readability and understanding.
- Remove unnecessary outputs before committing, and save final outputs in the correct project folders.
- Use relative paths such as data/raw/ and data/processed/; avoid hardcoded personal file paths.

- Set random seeds where possible, record package dependencies in requirements files such as requirements.txt, and do not commit passwords, API keys, private data or unnecessary large files.
- Use GitHub branches and pull requests for major changes. Commit messages must explain the change, for example Add pre-processing validation checks.
- Store model components, embeddings, evaluation metrics and experiment outputs in structured folders to support reproducibility and comparison between baseline, neural, and transformer models.

3. Data Quality Testing and Validation

Check	Purpose	Expected action/result
Schema	Confirm required fields: id, domain, difficulty, question and reference_answer.	Invalid records flagged; data dictionary updated.
Missing values	Find missing questions, reference answers, domains or difficulty labels.	Missing records removed, filled or documented.
Duplicates/leakage	Detect repeated or near-repeated Q&A pairs after normalisation.	Duplicates removed before final cleaned dataset.
Text quality	Identify malformed, too-short, too-long or encoding-damaged records.	Outliers reviewed and cleaning decisions saved.
Label consistency	Standardise domain and difficulty categories.	Final categories match the app role/domain mapping.
Pipeline run	Run the full pipeline from raw/interim data into processed outputs.	Non-empty questions_cleaned.json and questions_cleaned.csv produced.
Reproducibility	Confirm dependencies, file structure and outputs are restorable from repo instructions.	README/run instructions verified by a team member.

Table 2: Data Quality and Validation Checks

4. Automated and manual test plan

Unit tests will be written using pytest for reusable functions. The aim is to detect errors before full model training or app testing.

Area	Test Case	Expected result	Owner
Data ingestion	Load supported source formats and verify extracted question/answer records.	Records extracted without crashes; invalid sources produce clear errors.	Data/Model Lead
Transformation	Handle answer prefixes such as Answer:, answer:, ANSWER:, ans: and empty inputs.	Answers extracted consistently; empty inputs handled safely.	Data/Model Lead
Cleaning	Pass punctuation, casing, duplicate, blank, null and non-dict inputs through cleaning logic.	Cleaned values or safe validation messages returned.	Data/Model Lead
Pipeline	Run with missing/empty raw folder and with a small valid fixture dataset.	Missing data fails clearly; valid fixture produces cleaned outputs.	Data/Model Lead
Model scoring	Submit strong, partial, weak and blank answers to each scoring model.	Scores remain 0-100; relevant answers score higher than unrelated answers.	Model Lead / Pretrained Model Contributor
Feedback	Review feedback text for low, medium and high scoring answers.	Feedback is understandable, specific and actionable.	Test Plan Manager

API	Call roles, domains, models, start, submit, skip, repeat, completion, status and report endpoints.	Valid inputs return expected JSON; invalid inputs return safe 4xx messages.	App Developer
Frontend	Complete candidate journey through browser UI.	No broken controls; report appears after interview completion.	App Developer
Report delivery	Download PDF and test valid/invalid email delivery paths.	PDF downloads; invalid email rejected; missing SMTP credentials documented.	App Developer

Table 3. Pipeline Unit Test Plan

5. Model Testing Criteria

Model/test area	Metric or KPI	Target/evaluation method
TF-IDF + Grammar	Cosine/keyword similarity, grammar feature behaviour and output consistency.	Fast baseline; limitations in keyword matching documented.
Siamese Bi-LSTM	Checkpoint loading/training, score range, strictness with reworded answers and fixed curriculum behaviour.	Model runs without errors; scoring limitations stated clearly.
Sentence-BERT	Cosine similarity, dependency/model loading, semantic relevance checks.	Paraphrased relevant answers should generally score above unrelated answers.
Model selector	User chooses one model before interview starts and it remains locked for the session.	Report and UI show the selected scoring model correctly.
Comparison tool	Same answer scored across all three models using single-question and topic comparison views.	Scores generated without crashes; differences are explainable.
Fairness/validity limitation	Manual review of whether scores are reasonable and not overstated as hiring-grade.	Documentation states prototype scoring limits and need for human validation.

Table 4. Model Testing and Performance Evaluation Plan

Selection rule: The final model will not be selected by accuracy alone. The team will also consider interpretability, feedback usefulness, consistency, fairness of scoring and suitability for the mock interview app.

6. User Acceptance Testing/UAT Plan

Scenario	User action	Success criteria
Open app	Open the local or deployed main app URL.	Landing/chat interface loads with no visible errors.
Choose role	Select Data Scientist, Machine Learning Engineer, NLP Engineer, Data Analyst or Python Developer.	System starts the correct role-based interview and displays the first question.
Custom interview	Choose Custom Interview and select one or more domains.	Interview starts using selected domains; empty selection is prevented or handled clearly.
Choose model	Select Bi-LSTM, Sentence-BERT or TF-IDF before starting.	Selected model is locked for the interview and appears in final report.
Submit strong answer	Enter a relevant detailed technical answer.	Feedback is positive/useful and the app continues without errors.

Submit weak answer	Enter a vague or incorrect answer.	Feedback identifies improvement needs without crashing or using confusing wording.
Blank answer	Try submitting an empty response.	Validation message appears and interview state remains stable.
Repeat question	Click repeat during an active question.	Same question is displayed again; no duplicate scoring occurs.
Skip question	Skip one question.	Skip is recorded and the app moves forward cleanly.

Table 5: UAT Scenarios for Mock Interview App

UAT feedback form: Participants will rate ease of use, clarity of questions, relevance of scores, helpfulness of feedback, trust in readiness score and overall usefulness on a 1-5 scale, with one open comment box.

7. UAT Feedback Form

Question	**Rating scale / response**
The app was easy to understand and use.	1 Strongly disagree - 5 Strongly agree
The interview questions were clear.	1 Strongly disagree - 5 Strongly agree
The selected role or custom topic felt relevant.	1 Strongly disagree - 5 Strongly agree
The feedback after answers was useful.	1 Strongly disagree - 5 Strongly agree
The final report was clear and helpful.	1 Strongly disagree - 5 Strongly agree
The score/readiness result felt trustworthy for practice purposes.	1 Strongly disagree - 5 Strongly agree
The PDF/email report actions worked as expected.	Yes / No / Not tested
Would you use this app again for interview practice?	Yes / Maybe / No
Most useful part of the app	Open comment
Most confusing part or suggested improvement	Open comment

Table 6: UAT feedback form

Review process: Each major change is completed on a branch, submitted through a pull request, checked by at least one reviewer, corrected if needed, then merged only after approval. Components are signed off when code runs without errors, tests pass, outputs are saved correctly, results are documented, and the Team Leader confirms readiness.

Final testing summary:

This final plan verifies the complete workflow from data preparation and model scoring through the working Flask application, browser interview experience, final report delivery and UAT sign-off. The system should be presented as a deployable prototype for interview practice, not as a production hiring assessment tool. Final readiness depends on both technical smoke-test evidence and user feedback from UAT participants.